



Secured Application Development

APIs and Endpoints



Secured Application Development

Dr. Abdullmalek Alqobaty

**Associated professor
Computer Engineering**

Secured Application Development

APIs and Endpoints



APIs and Endpoints

APIs and Endpoints



APIs and Endpoints

APIs and Endpoints

- ❑ In this lecture, we will begin looking at coding web applications, starting with designing our endpoints: **URLs and APIs.**
- ❑ URLs and APIs are the building blocks of a web application. HTTP leaves a number of choices to us:
 - what request method to use?
 - what response code to return?
 - what format to use for the request and response body?
- ❑ We will begin by looking at the anatomy of a URL before exploring REST APIs. These are a specific type of API that leverages the HTTP protocol to enable stateless requests to server-side functionality.



APIs and Endpoints

APIs and Endpoints

- ❑ **A key method for restricting access to server-side functionality is username and password-based authentication.**
 - We will begin looking at this as well as how to use unit testing to ensure permissions are set up correctly. (topic it is covered more fully later on)
 - We finish by looking at some attacks that exploit vulnerabilities in APIs: **deserialization attacks.**



APIs and Endpoints

URLs

- ❑ The general form of a URL is

schema://user:password@host:port/path?query#fragment

- All parts are optional except path.
 - **If path begins with a slash /, then it is an absolute path; otherwise, it is relative to the URL it is loaded from.**
- ❑ This is the URL in a link or the address bar of a browser. We saw that the actual request sent once the connection is established

GET path?query HTTP/1.1

Host: host@port

- ❑ The fragment (**the part after the #**) is not sent but applied by the browser.
- ❑ The **username and password** are placed **in a header**, which we discuss later.



APIs and Endpoints

URLs

- ❑ If HTTPS is used, the entire request is encrypted.
 - The host and port are sent unencrypted over the network so that the connection can be established.
 - The query parameters, username, and password are not.
- ❑ Despite this, we should not send sensitive data in a GET request. The reasons are as follows:
 - The URL is stored in the browser history.
 - The URL may be logged on the server and remain in backups, including on other servers such as proxies.
 - GET requests should not change state, as we will see later.
- ❑ We use the POST method when sensitive data is sent to the server. Sensitive data includes usernames, passwords, authorization tokens, session IDs, as well as any other secret data such as credit card details.



APIs and Endpoints

REST APIs

- ❑ REST (Representational State Transfer) APIs provide users and applications with programmatic access to functionality.
 - Instead of returning HTML pages, they return data, serialized for transport in some way as JSON or XML.
- ❑ REST APIs has one difference from HTTP requests **only the data type of the body**. They use same HTTP methods (GET, POST, PUT, etc.) and the same response codes (200 OK, 404 Not Found, etc.).
- ❑ A misconception is that the term “REST” applies to any API **that allows programmatic access to the application over HTTP**.
- ❑ By keeping to the **correct REST principles**, we avoid some common vulnerabilities. We can , also, use **frameworks** that take care of the mechanics of REST, allowing us to focus on business logic. **We write less code, and it is clearer.**
- ❑ REST APIs **operate on items and collections**. An **item** is a single entity, such as an address and a **collection** is a group of items, such as an address book.



APIs and Endpoints

GET Requests

- ❑ REST GET requests can be called
 - on an item, for example:
<http://api.example.com/addresses/100>
 - or on a collection, for example:
<http://api.example.com/addresses>
 - In the former, a single address, with an ID of 100, is returned. In the latter, a set of addresses is returned.
- ❑ A GET request is idempotent: it is safe to call it more than once, and it should return the same result each time.
 - It is cacheable: a browser can store the result rather than making a fresh request to the server each time it is requested.
 - GET requests should not change state, both out of idempotency and security reasons.
- ❑ The return status should be 200 OK if the requested item exists, with the item or collection in the body, and 404 Not Found otherwise.



APIs and Endpoints

POST Requests

- ❑ POST is for creating an item.
 - It is therefore only called on collections.
 - It is not idempotent: if you call it more than once, a new item will be created each time.
 - It should not be cached.
- ❑ Returning a response body is optional. You may return the ID of the newly created item, in which case the response code should be 201 Created or 200 OK.
- ❑ If you do not return the ID or item, return a response code 204 No Content.



APIs and Endpoints

PUT Requests

- ❑ **PUT is for updating an existing item.** It is therefore only called on an item.
- ❑ **PUT requests are not cached.** To see why, imagine you update a record, say, an address, using PUT. Then, another user updates the same record. If you want to change it back, you would issue the same PUT request as before. **If it were cached, it would not execute on the server.**
- ❑ **PUT is regarded as idempotent, as calling it multiple times has the same effect.** It should return.
 - 200 OK if the response contains the ID or item,
 - 204 No Content if it does not, or
 - 404 Not Found if the record does not exist.



APIs and Endpoints

PATCH Requests

- ❑ PATCH, like PUT, updates a record, so it is called on an item. **However, PUT is a full update, whereas PATCH is a partial update.** You only supply PATCH with the data that are changing, not the whole record.

- ❑ Unlike PUT, PATCH is not idempotent. To see why, imagine we have the address in our database:

Column	Value
Street	1060 W Addison St
City	Chicago
State	IL
Postcode	60613

- ❑ Say we use PATCH to set the postcode to 60600. If we use JSON, the request might look like

```
{  
  "Postcode": "60600"  
}
```

The address in the database would become

Column	Value
Street	1060 W Addison St
City	Chicago
State	IL
Postcode	60600

- ❑ PATCH should return 200 OK if the response contains the ID or item, 204 No Content if it does not, or 404 Not Found if record does not exist.



APIs and Endpoints

DELETE Requests

- ❑ **DELETE removes an item, so it is called on items, not collections.** It is normally regarded as idempotent as deleting a record a number of times makes the database look the same.
 - However, deleting a nonexistent record results in a 404 Not Found message, so arguably it is not idempotent at all.
- ❑ **DELETE is not cacheable as we might wish to call DELETE, then a POST to create the item again, then another DELETE.**
- ❑ **DELETE should return 200 OK if the response contains a body** (e.g., a status message or the ID of the deleted object).
 - It should return 204 No Content if the response does not contain a body and
 - 404 Not Found if the item does not exist.



APIs and Endpoints

Unit Testing Permissions

- ❑ One common mistake is **to not apply permission checks on URLs** called internally.
 - For example, imagine a user has entered form data. The Submit button sends them to the URL/`submitform`. **Once the form has been processed, the user is redirected to a URL/`viewdata` that displays the data they submitted.**
 - The `/viewdata` function needs the same permission checks as `/submitform`; **otherwise, a malicious user could visit it directly.**
- ❑ In case errors do slip through, we can unit test permissions. **Good developers unit test the functionality of their code but can forget to test that the code is callable by valid users and not callable by invalid ones.**
- ❑ We should also check that it can be called when a user is logged in and cannot be called when there is no logged-in user.



APIs and Endpoints

Unit Testing Permissions (Cont.)

Let's take an example,

- ❑ for function-based views, Django adds clarity to permissions. When a page should only be viewable by a logged-in user, **we can precede the function with required login.**
- ❑ If a user is logged in, the function is executed. If not, the user is redirected to the login page and sent back upon successful login. e. g. Django has a **@permission_required decorator** that returns an error if the logged-in user doesn't have the stated permission.
- ❑ Using permission classes in REST view classes adds clarity to your code, making it easier to spot incorrect authorization settings.



APIs and Endpoints

Deserialization Attacks

- ❑ JSON is used for request and response bodies. JSON has become a popular serialization format for a number of reasons:
 1. It is easy for humans to read and edit with standard editors.
 2. It works for many languages.
 3. It is fairly compact.
 4. Its functionality is limited, minimizing risks.
- ❑ XML was a popular format before JSON gained popularity and still is, especially in older frameworks.
- ❑ Data formats should be chosen carefully as they can lead to vulnerabilities.
- ❑ A malicious user can craft requests that exploit vulnerabilities when deserialized at the server. An attacker can stage a man-in-the-middle attack to intercept and alter request and response bodies between the server and another user. **If the deserialization is not secure, undesirable actions can be performed on the server.**



APIs and Endpoints

XML Attacks

- ❑ XML is a flexible data format that contains features developers are sometimes unaware of and can be exploited.
- ❑ A famous XML vulnerability is known as the **Billion Laughs attack**.
- ❑ XML supports a directive called `<!ENTITY>`.
- ❑ `<!ENTITY>` allows the author of the XML to define a shortcut. For example, the author could define

`<!ENTITY ms "Microsoft Corp">`

- ❑ Then, instead of writing Microsoft Corp in the XML body, the author just has to write `&ms`.
- ❑ Unfortunately, **entities are allowed to be recursive**: It is exploited by the **Billion Laughs attack**.



APIs and Endpoints

❑ Consider the following XML document.

```
<?xml version="1.0"?>
<!DOCTYPE hahas [
<!ENTITY haha "haha">
<!ENTITY haha2 "&haha;&haha;&haha;&haha;&haha;&haha;&haha;&haha;&haha;">
<!ENTITY haha3 "&haha2;&haha2;&haha2;&haha2;&haha2;&haha2;&haha2;&haha2;&haha2;">
<!ENTITY haha4
"&haha3;&haha3;&haha3;&haha3;&haha3;&haha3;&haha3;&haha3;">
<!ENTITY haha5
"&haha4;&haha4;&haha4;&haha4;&haha4;&haha4;&haha4;&haha4;">
<!ENTITY haha6
"&haha5;&haha5;&haha5;&haha5;&haha5;&haha5;&haha5;&haha5;">
<!ENTITY haha7
"&haha6;&haha6;&haha6;&haha6;&haha6;&haha6;&haha6;&haha6;">
<!ENTITY haha8 "&haha7;&haha7;&haha7;&haha7;&haha7;&haha7;&haha7;&haha7;">
<!ENTITY haha9
"&haha8;&haha8;&haha8;&haha8;&haha8;&haha8;&haha8;&haha8;">
]>
<hahas>&haha9;</hahas>
```



APIs and Endpoints

Billion Laughs Attack

- ❑ As a result, placing a single **&haha9;** in the last line of the document results in the insertion of one billion haha's. A file that has taken no more than 1K of text for the attacker **has consumed 3GB of memory on the server.**
- ❑ Of course, no developer would write such code in their own application.
- ❑ However, if server-side code to parse XML exists and entity expansion is allowed, as is the case in many XML parsers, then an **attacker can craft a request** that results in a DoS.
- ❑ The entity tag also supports a parameter called SYSTEM. This expands to the contents of a URL.
- ❑ **It can be exploited by the XML External Entity (XXE) attack, resulting in file disclosure if the expanded XML is visible to the attacker.**



APIs and Endpoints

Billion Laughs Attack

- ❑ The listing below displays the Unix password file.

```
<?xml version="1.0"?>
<!DOCTYPE foo [
  <!ELEMENT foo ANY >
  <!ENTITY xxe SYSTEM "file:///etc/passwd" >
]>
<foo>&xxe;</foo>
```

Function Calls and Creation

- ❑ Some serialization protocols allow functions or classes to be created from a serialized stream.
- ❑ One example is the BinarySerializer from Microsoft's .NET framework. *While not allowing arbitrary code to be created, it allow dangerous .NET functionality to be called.*
- ❑ Microsoft now recommends against its use.



APIs and Endpoints

XML External Entity Attacks

- ❑ A document type definitions (DTD) can include content from external files. If an XML parser is configured to process inline DTDs, **an attacker can use these external entity declarations to explore the local filesystem or to trigger network requests from the web server itself.**
- ❑ A typical external entity looks like the following.

```
<?xml version="1.0" standalone="no"?>
<!DOCTYPE copyright [
<!ELEMENT copyright (#PCDATA)>
<!ENTITY copy PUBLIC "http://www.w3.org/xmlspec/copyright.xml">
]>
<copyright> &copy; </copyright>
```



APIs and Endpoints

XML External Entity Attacks

- ❑ According to the XML 1.0 specification, a parser is expected to read the contents of the file specified in the external entity and insert that data into the XML document wherever the entity is referenced.
- ❑ In this example, the data hosted at <http://www.w3.org/xmlspec/copyright.xml>, would be inserted into the XML document wherever the text © appears.
- ❑ The URL referenced by the external entity declaration can use various network protocols, depending on the prefix. Our example DTD uses the `http://` prefix, which will cause the parser to make an HTTP request.
- ❑ The XML specification also supports reading local files on disk, using the `file://` prefix. **For this reason, external entity definitions are a security disaster.**



APIs and Endpoints

Defending Against Deserialization Attacks

The best defense against these attacks is restricted deserialization:

1. Don't allow function creation .
2. Don't allow function calls, or at least severely restrict them.
3. Understand your deserialization tools, especially XML parsers.
4. Sanitize the body and ensure it complies with the format specifications.
5. Avoid descriptive error messages.



APIs and Endpoints

How Hackers Exploit External Entities

- ❑ When an XML parser throws an error, the error message will often include the contents of the XML document being parsed. **Knowing this, hackers use external entity declarations to read files on a server.**
- ❑ **A maliciously crafted XML file might include a reference to a file such as `file:///etc/passwd` on a Linux system, for instance.** When this external file is inserted into the XML document by the parser, the XML becomes malformed-so parsing fails.
- ❑ **The parser then dutifully includes the contents of the file in the error response, allowing the hacker to view the sensitive data within the referenced file.**
- ❑ Using this technique, hackers can read sensitive files on a vulnerable web server that contain passwords and other confidential information.



APIs and Endpoints

How Hackers Exploit External Entities

- ❑ External entities can also be used to commit server-side request forgery (SSRF) attacks.
- ❑ When a web server makes outgoing HTTP requests, and a hacker can control which URLs those requests are sent, you are vulnerable to a server-side request forgery (SSRF) attack, and a hacker can use your server to send malicious requests.
 - One of the reasons to make outbound network requests from the server, using any kind of third-party API.
- ❑ A hacker will check a site for SSRF vulnerabilities by spidering through it, navigating to every page, and using hacking tools to replace every HTTP parameter they encounter with a URL under their control.
 - If they detect any HTTP requests to their trap URL, they know the requests must have been triggered from your server, and that you are vulnerable to SSRF.
- ❑ Hackers will also check to see if any part of your site accepts XML content, and use XML external entity attacks in an attempt to commit SSRF.
- ❑ Chapter 15 discussed this attack vector.



APIs and Endpoints

Protecting Against Server-Side Forgery

- ❑ **You can protect yourself against server-side forgery at several levels.**
 - The first, and most important step, is to audit any parts of your code that make outbound HTTP requests.
 - You will almost know ahead of time which domains need to be invoked as part of API calls, **so the construction of URLs for API calls should use web domains recorded in your configuration or code rather than coming from the client.**
 - One way of ensuring this is to use the software development kit (SDK) that is usually made freely available with most APIs.
- ❑ Because you should be following the practice of defense in depth, **it makes sense to install safeguards against SSRF at the network level.**



APIs and Endpoints

Protecting Against Server-Side Forgery

- ❑ **Whitelisting the individual domains that you need access to in your firewall, and banning all others, is a good way to catch any security issues you may have overlooked during code review.**
- ❑ **Finally, consider employing penetration testing to detect SSRF vulnerabilities in your code.**
 - **This can be done by employing an external team to find vulnerabilities in your website or by using an automated online tool to do the same.**
 - **Effectively, you will be using the same tools that hackers use to detect vulnerabilities, before they get the chance to do so themselves.**



APIs and Endpoints

Function Calls and Creation

- ❑ JSON is regarded as a safe format if sensibly used. It only supports simple data types (strings, integers, booleans, etc.), arrays, and dictionaries. **It does not support function creation or calls.**
- ❑ The way in which the deserialized data are used may introduce vulnerabilities, even with JSON.
- ❑ **Before deserializing, check if the size is within bounds to avoid memory issues.**
- ❑ When a deserialization error does take place, **avoid telling the user what went wrong.** This can enable the attacker to discover vulnerable parts of your code.